



Universidad Católica de Honduras
"NUESTRA SEÑORA REINA DE LA PAZ"

Seminario – Taller de Software

Documentacion Tecnica

Casa Latina Smart System

Catedrática

Ing. Reina Tosta

Integrantes

Fabricio Antonio Rodriguez

Jose Mario Mejia

Fecha

4 de agosto de 2026

Tabla de contenido

1. Introducción

2. Descripción general del sistema

3. Arquitectura y estructura de archivos

3.1 Estructura de carpetas

3.2 Patrón general de cada módulo

4. Tecnologías utilizadas

5. Roles y control de acceso

5.1 Diferencias de permisos dentro de un mismo módulo

5.2 Reglas especiales de la administración de usuarios

6. Modelo de datos

6.1 Diagrama de relaciones (resumen textual)

6.2 Diccionario de datos

7. Módulos del sistema

7.1 Autenticación

7.2 Panel principal / Dashboard

7.3 Mesas y pedidos activos

7.4 Pedido — Paso 1: Mesa y productos

7.5 Pedido — Paso 2: Revisión y envío a cocina

7.6 Pedido — Paso 3: Cobro

7.7 Menú

7.8 Stock / Inventario

7.9 Compras

7.10 Proveedores

7.11 Gastos

7.12 Factura

7.13 Reportes

7.14 Usuarios y Empleados

7.15 Mi Perfil

7.16 Auditoría

8. Flujo completo de un pedido

9. Generación de documentos PDF

10. Reglas de negocio transversales

11. Diseño visual

12. Requisitos e instalación

13. Pruebas del sistema

13.1 Ad hoc testing con criterio

13.2 Pruebas de caja negra

13.3 Conclusión de las pruebas de caja negra

14. Reporte de Cambios — Antes vs. Ahora, por función

- 14.1 Pedidos
- 14.2 Menú
- 14.3 Stock / Inventario
- 14.4 Compras
- 14.5 Proveedores
- 14.6 Factura, Recibo, Comanda y División de Cuenta
- 14.7 Gastos
- 14.8 Reportes
- 14.9 Dashboard (Inicio)
- 14.10 Usuarios, Mi Perfil y Auditoría (módulos nuevos)
- 14.11 Diseño Visual y Seguridad

15. Diagrama de Gantt del Proyecto

1. Introducción

Casa Latina Smart System es una aplicación web desarrollada en PHP para la administración integral de un restaurante de comida típica hondureña. El sistema cubre el ciclo completo de operación del negocio: toma de pedidos por mesa, envío a cocina por comandas, cobro y facturación, control de inventario con descuento automático por receta, compras a proveedores, registro de gastos, generación de reportes financieros en PDF, administración de usuarios y empleados, y una bitácora de auditoría de todas las acciones relevantes.

Este documento describe la arquitectura, el modelo de datos, los módulos funcionales y las reglas de negocio del sistema, tomando como base el código fuente y los comentarios incluidos por los propios desarrolladores en cada archivo.

2. Descripción general del sistema

El sistema está organizado en módulos independientes (un archivo PHP por pantalla), todos protegidos por una capa de autenticación y control de acceso por rol. Los módulos se comunican entre sí mediante una base de datos relacional (MariaDB/MySQL) y comparten un layout visual común (barra superior, menú lateral y paleta de colores configurable).

Los procesos centrales del sistema son:

- Gestión de mesas y pedidos, con un flujo de 3 pasos: armar pedido → revisar y enviar a cocina → cobrar.
- Control de inventario (stock de insumos) con descuento automático cuando se envían platos a cocina, según la receta configurada en el módulo Menú.
- Compras a proveedores, que alimentan directamente el stock.
- Facturación automática al momento de cobrar un pedido (no se crean facturas a mano).
- Generación de documentos imprimibles en PDF: factura, recibo, comanda de cocina, división de cuenta y ficha de receta.
- Registro de gastos operativos y fijos del negocio, organizados por categoría.
- Reportes financieros y operativos (ventas, gastos, compras, utilidad, platos más vendidos, cierre de caja, inventario, auditoría) exportables a PDF.
- Administración de usuarios, empleados y una bitácora de auditoría de solo lectura.
- Un espacio de "Mi Perfil" donde cualquier usuario puede cambiar su propia contraseña sin depender de un administrador.

3. Arquitectura y estructura de archivos

El proyecto sigue una arquitectura de páginas PHP procedurales (sin framework), donde cada archivo en la raíz del proyecto representa una pantalla del sistema. La lógica compartida (conexión a base de datos, autenticación, auditoría y layout visual) vive en la carpeta includes/, y las dependencias externas se gestionan con Composer en vendor/.

3.1 Estructura de carpetas

```
CASALATINA/  
├── assets/  
│   └── style.css
```

Hoja de estilos única del sistema (paleta de colores centralizada)

includes/	
auth.php	Sesión y control de acceso por rol
db.php	Conexión PDO a la base de datos
auditoria.php	Función registrarAuditoria() usada por todos los módulos
layout_top.php	Cabecera HTML común, menú lateral dinámico según rol
layout_bottom.php	Cierre de las etiquetas HTML abiertas en layout_top.php
tema.php	Constantes PHP con la paleta de colores (para los PDF)
vendor/	Dependencias de Composer (dompdf/dompdf, para generar PDFs)
composer.json / composer.lock	
FCLschema.sql	Esquema de la base de datos (estructura de tablas)
datos_prueba.sql	Datos de ejemplo adicionales
login.php / logout.php	Autenticación
index.php	Panel principal (dashboard)
pedidos_listado.php	Mapa de mesas y pedidos activos
pedido_paso1.php	Pedido – Paso 1: mesa y productos
pedido_paso2.php	Pedido – Paso 2: revisión y envío a cocina
pedido_cobro.php	Pedido – Paso 3: cobro
menu.php	Catálogo de platos/bebidas y recetas
factura.php	Historial y búsqueda de facturas
stock.php	Inventario de insumos
compras.php	Registro de compras a proveedores
proveedores.php	Catálogo de proveedores
gastos.php	Categorías y detalle de gastos
reportes.php	Selector de reportes
usuarios.php	Administración de empleados y usuarios del sistema
perfil.php	Mi Perfil: cambiar la propia contraseña
auditoria.php	Bitácora de auditoría (solo administrador)
factura_pdf.php, recibo_pdf.php, comanda_pdf.php, division_pdf.php, reporte_pdf.php, receta_pdf.php	(generadores de PDF con Dompdf)

3.2 Patrón general de cada módulo

La gran mayoría de los archivos de módulo siguen el mismo patrón, visible directamente en su estructura:

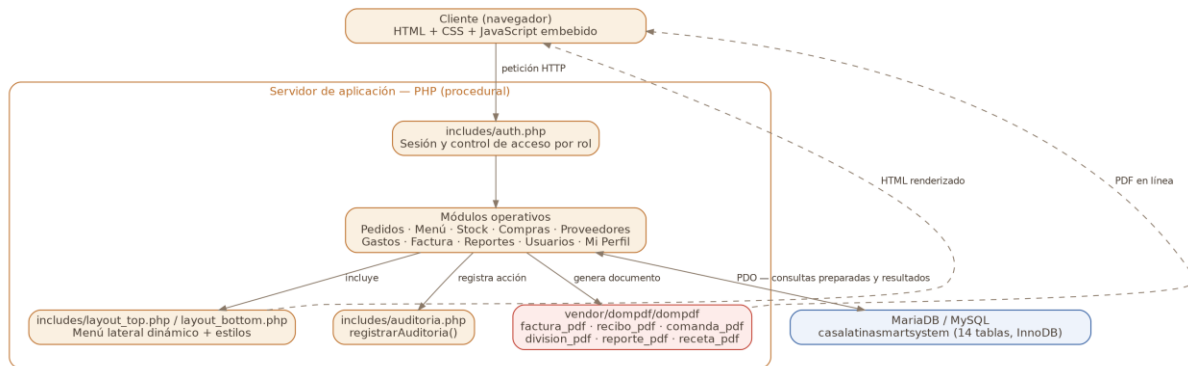
1. Definir `$modulo_actual` e incluir `includes/auth.php` (control de acceso) e `includes/db.php` (conexión).
2. Procesar el formulario POST si corresponde (crear, editar, eliminar/desactivar), usualmente dentro de una transacción PDO cuando hay varias tablas involucradas.
3. Registrar la acción en la auditoría mediante `registrarAuditoria()`, siempre después de que la operación ya se ejecutó con éxito.
4. Consultar los datos a mostrar (SELECT) e incluir `includes/layout_top.php`.
5. Renderizar el HTML de la pantalla (con estilos locales embebidos con prefijo `pd-`).
6. Incluir JavaScript embebido para búsquedas en vivo, cálculos en pantalla y armado de formularios dinámicos (por ejemplo, listas de ítems de un pedido o ingredientes de una receta) antes de enviarlos como JSON oculto.
7. Cerrar con `includes/layout_bottom.php`.

4. Tecnologías utilizadas

Componente	Tecnología
Lenguaje del servidor	PHP (estilo procedural, con PDO para acceso a datos)
Base de datos	MariaDB / MySQL (utf8mb4, motor InnoDB con llaves foráneas)

Componente	Tecnología
Generación de PDF	Dompdf (paquete dompdf/dompdf, instalado vía Composer en vendor/)
Gestión de dependencias	Composer (composer.json / composer.lock)
Frontend	HTML + CSS propio (assets/style.css) + JavaScript embebido sin frameworks
Sesiones	Sesiones nativas de PHP (session_start(), \$_SESSION)
Contraseñas	password_hash() / password_verify() (bcrypt) — con compatibilidad hacia texto plano en login.php
Identidad visual	Marca de texto ("CASA LATINA") definida en CSS — el sistema ya no depende de un archivo de imagen de logo

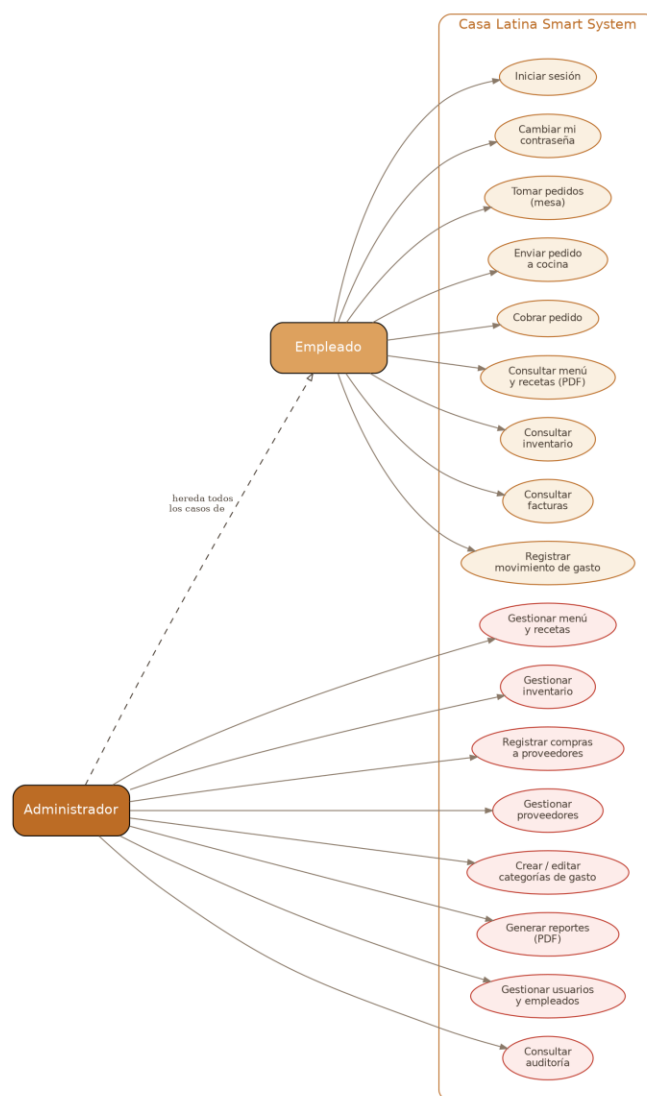
El acceso a la base de datos se realiza siempre mediante PDO con sentencias preparadas, tanto para las consultas de lectura como para las inserciones/actualizaciones, lo que protege al sistema de inyección SQL.



5. Roles y control de acceso

El sistema maneja dos roles de usuario, definidos en la tabla usuarios: administrador y empleado. El control de acceso se resuelve en includes/auth.php: cada página define la variable \$modulo_actual antes de incluir ese archivo, y auth.php compara ese módulo contra una lista fija de módulos permitidos para el rol empleado (\$modulos_empleado). Si el módulo no está en esa lista y el usuario no es administrador, se redirige a index.php.

Los módulos accesibles para el rol empleado son: inicio, menú, pedido, factura, stock, gastos y perfil ("Mi Perfil"). Todos los demás módulos (compras, proveedores, reportes, usuarios y auditoría) son exclusivos del rol administrador.



Además del control centralizado en auth.php, los módulos usuarios.php y auditoria.php incluyen una verificación adicional de rol al inicio del archivo, como defensa extra por si en el futuro se agregara por error ese módulo a la lista de módulos de empleado.

5.1 Diferencias de permisos dentro de un mismo módulo

Algunos módulos son visibles para ambos roles, pero con distinto nivel de permiso dentro de la misma pantalla:

Módulo	Empleado	Administrador
Menú	Solo lectura (puede ver e imprimir la receta en PDF)	Crear, editar, desactivar/reactivar platos y su receta
Stock	Solo lectura	Crear, editar, desactivar/reactivar productos
Gastos	Puede registrar detalles de gasto (movimientos)	Además puede crear y editar categorías de gasto
Pedido / Factura	Acceso completo (operación diaria de caja)	Acceso completo

Módulo	Empleado	Administrador
Mi Perfil	Puede cambiar su propia contraseña	Puede cambiar su propia contraseña (no la de otros — para eso está Usuarios)

5.2 Reglas especiales de la administración de usuarios

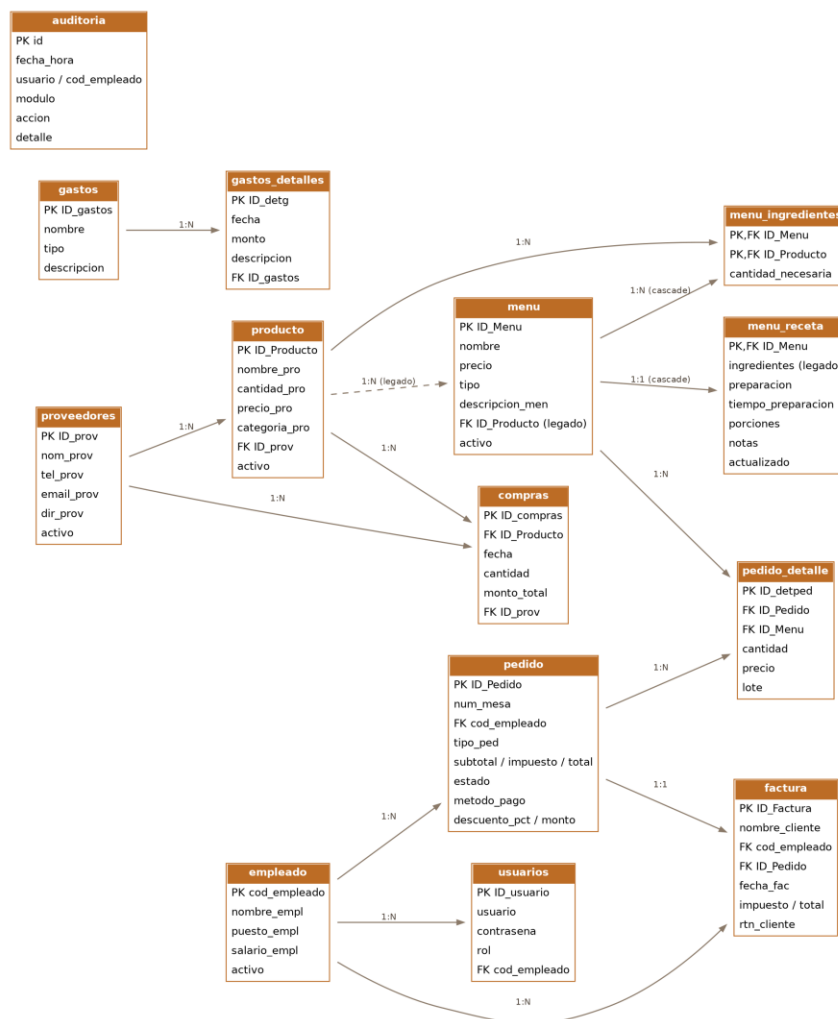
- No se puede quitar el rol de administrador al último usuario administrador del sistema (función `contarAdmins()` en `usuarios.php`).
- No se puede eliminar el último usuario administrador.
- Un usuario no puede eliminar su propia cuenta mientras tiene la sesión iniciada.
- Las contraseñas nunca se muestran de vuelta en los formularios de edición: el campo se deja en blanco y solo se actualiza si se escribe una nueva.
- Cambiar la propia contraseña desde Mi Perfil exige escribir la contraseña actual (para confirmar identidad), un mínimo de 6 caracteres, que la confirmación coincida, y que la nueva contraseña sea distinta a la actual.

6. Modelo de datos

La base de datos `casalatinasmartsystem` está definida en `FCLschema.sql`, con 14 tablas relacionadas mediante llaves foráneas. A continuación se describe cada tabla con sus campos principales, tal como están definidos en el esquema, más el propósito de negocio que cumplen según el uso que les da el código.

6.1 Diagrama de relaciones (resumen textual)

- proveedores → producto (un proveedor surte muchos productos/insumos)
- producto → compras (un producto tiene muchas compras registradas)
- proveedores → compras (una compra puede o no tener proveedor asociado)
- producto → menu (campo heredado `ID_Producto`; el descuento real de stock usa `menu_ingredientes`, no esta relación)
- menu ↔ producto → menu_ingredientes (receta: qué insumos y cuánto necesita cada plato para descuento automático de stock)
- menu → menu_receta (preparación, tiempo, porciones y notas en texto libre — independiente del descuento de stock)
- empleado → pedido, factura, usuarios (un empleado puede tener pedidos, facturas y una cuenta de usuario asociada)
- pedido → pedido_detalle (líneas de productos de un pedido, agrupadas por lote/ronda de envío a cocina)
- pedido → factura (relación 1 a 1: cada pedido cobrado genera exactamente una factura)
- gastos → gastos_detalle (una categoría de gasto tiene muchos movimientos, cada uno con su propia descripción)
- auditoria es una tabla independiente, de solo inserción, que registra usuario, empleado, módulo, acción y detalle de cada evento relevante.



6.2 Diccionario de datos

proveedores

Catálogo de proveedores de insumos. Se desactivan (activo=0) en vez de borrarse si tienen productos asignados, para no perder esa referencia.

Campo	Tipo	Descripción
ID_prov	varchar(10)	Llave primaria. Formato PVxxxx generado por proveedores.php.
nom_prov	varchar(10)	Nombre del proveedor.
tel_prov	int(11)	Teléfono de contacto.
email_prov	varchar(100)	Correo electrónico, método de contacto alternativo.
dir_prov	varchar(30)	Dirección / bodega.
activo	tinyint(1)	1 = activo, 0 = desactivado (baja lógica).

producto

Inventario de insumos de cocina (stock). Se desactiva en vez de borrarse si tiene compras o recetas asociadas.

Campo	Tipo	Descripción
ID_Producto	varchar(10)	Llave primaria (ej. PR001).
nombre_pro	varchar(20)	Nombre del insumo.
cantidad_pro	decimal(10,2)	Cantidad disponible en stock. Se actualiza automáticamente por Compras y por el descuento de recetas.
precio_pro	decimal(6,2)	Precio de referencia (costo).
categoria_pro	varchar(50)	Categoría, elegida de una lista de 10 categorías estándar (o texto libre si se elige "Otros").
ID_prov	varchar(10)	Proveedor asignado (FK a proveedores, opcional).
activo	tinyint(1)	1 = activo, 0 = desactivado.

compras

Historial de compras de insumos a proveedores. Cada compra registrada suma automáticamente al stock del producto comprado (compras.php).

Campo	Tipo	Descripción
ID_compras	varchar(10)	Llave primaria (ej. C000001).
ID_Producto	varchar(10)	Insumo comprado (FK a producto).
fecha	date	Fecha de la compra (fijada al día actual por el servidor, no editable desde el formulario).
cantidad	decimal(10,2)	Cantidad comprada.
monto_total	decimal(8,2)	Monto total pagado.
ID_prov	varchar(10)	Proveedor de esa compra específica (FK a proveedores, opcional).

empleado

Personal del restaurante. Se desactiva en vez de borrarse si tiene pedidos, facturas o una cuenta de usuario asociada.

Campo	Tipo	Descripción
cod_empleado	varchar(10)	Llave primaria (ej. EMP01).
nombre_empl	varchar(30)	Nombre completo.
puesto_empl	varchar(20)	Puesto, elegido de una lista estandarizada (Administrador, Gerente, Cajero, Mesero, Cocinero, Ayudante de Cocina, Bodega/Inventario, Repartidor, Limpieza, u "Otro" con texto libre).
salario_empl	int(11)	Salario (opcional).

Campo	Tipo	Descripción
activo	tinyint(1)	1 = activo, 0 = desactivado.

usuarios

Cuentas de acceso al sistema, vinculadas opcionalmente a un empleado.

Campo	Tipo	Descripción
ID_usuario	varchar(10)	Llave primaria (ej. U001).
usuario	varchar(20)	Nombre de usuario (login).
contrasena	varchar(255)	Hash bcrypt (password_hash); login.php admite además comparación en texto plano por compatibilidad con datos antiguos.
rol	varchar(20)	administrador o empleado.
cod_empleado	varchar(10)	Empleado vinculado (FK a empleado, opcional).

menu

Catálogo de platillos y bebidas que se ofrecen a la venta.

Campo	Tipo	Descripción
ID_Menu	varchar(10)	Llave primaria, definida a mano al crear el ítem (ej. M001); se valida que no exista ya antes de insertar.
nombre	varchar(30)	Nombre del plato o bebida.
precio	decimal(8,2)	Precio de venta.
tipo	varchar(15)	Platillo o Bebida.
descripcion_men	varchar(50)	Descripción corta.
ID_Producto	varchar(10)	Campo heredado, no usado por el flujo de recetas actual (queda NULL al crear).
activo	tinyint(1)	1 = activo, 0 = desactivado (se desactiva si ya se usó en un pedido).

menu_ingredientes

Receta de descuento de stock: qué insumos y en qué cantidad consume cada plato al enviarse a cocina. Llave primaria compuesta (ID_Menu, ID_Producto). Se elimina en cascada si se borra el plato del menú. Esta es la única fuente de ingredientes: se usa tanto para el descuento de inventario como para la ficha de receta impresa.

Campo	Tipo	Descripción
ID_Menu	varchar(10)	Plato (FK a menu, ON DELETE CASCADE).
ID_Producto	varchar(10)	Insumo requerido (FK a producto).
cantidad_necesaria	decimal(8,2)	Cantidad del insumo que se descuenta por cada unidad vendida del plato.

menu_receta

Preparación de cocina en texto libre (pasos, tiempo, porciones y notas), independiente de menu_ingredientes. El campo ingredientes queda como legado del diseño anterior y ya no se llena desde el formulario actual — la lista de ingredientes vigente es siempre menu_ingredientes.

Campo	Tipo	Descripción
ID_Menu	varchar(10)	Llave primaria y FK a menu (ON DELETE CASCADE).
ingredientes	text	Campo heredado; ya no se escribe desde el formulario de Menú.
preparacion	text	Pasos de preparación.
tiempo_preparacion	varchar(30)	Ej. "20 min".
porciones	varchar(20)	Ej. "1 persona".
notas	text	Notas adicionales.
actualizado	datetime	Fecha/hora de la última modificación (automática).

pedido

Encabezado de cada pedido, con su ciclo de vida completo: Abierto → EnCocina → Pagado (o Cancelado). Todo pedido nuevo se guarda como tipo "Mesa" — la opción de "Envío" se eliminó del flujo de captura.

Campo	Tipo	Descripción
ID_Pedido	varchar(10)	Llave primaria (ej. P000001).
num_mesa	int(11)	Número de mesa, elegido de un desplegable que solo muestra mesas libres.
cod_empleado	varchar(10)	Cajero/mesero asignado (FK a empleado); se toma siempre de la sesión, nunca de un campo editable del formulario.
tipo_ped	varchar(15)	Siempre "Mesa" en pedidos nuevos (valor fijo en el código).
fecha	date	Fecha de creación del pedido.
subtotal	decimal(8,2)	Suma de cantidad × precio de todas las líneas. Se recalcula en el servidor (recalcularTotalesPedido).
impuesto	decimal(8,2)	15% sobre el subtotal con descuento aplicado.
total	decimal(8,2)	Subtotal – descuento + impuesto.
monto_recibido	decimal(8,2)	Efectivo recibido del cliente (pago en efectivo).
cambio	decimal(8,2)	Vuelto entregado.
estado	varchar(15)	Abierto, EnCocina, Pagado o Cancelado.
metodo_pago	varchar(20)	Efectivo o Tarjeta.
descuento_pct	decimal(5,2)	Porcentaje de descuento aplicado al cobrar (0–100).
descuento_monto	decimal(8,2)	Monto de descuento resultante.

pedido_detalle

Líneas de productos de un pedido. El campo lote agrupa los productos por ronda de envío a cocina, lo que permite reimprimir comandas parciales y agregar rondas nuevas a un pedido ya en cocina.

Campo	Tipo	Descripción
ID_detped	varchar(10)	Llave primaria (ej. D000001).
ID_Pedido	varchar(10)	Pedido al que pertenece (FK a pedido).
ID_Menu	varchar(10)	Plato/bebida pedido (FK a menu).
cantidad	int(11)	Cantidad pedida de ese ítem.
precio	decimal(8,2)	Precio del ítem al momento de agregarse (se congela, no cambia si luego cambia el precio del menú).
lote	int(11)	Número de ronda de envío a cocina (1, 2, 3...).

factura

Comprobante fiscal generado automáticamente al cobrar un pedido. Relación 1 a 1 con pedido. No se permite borrar facturas (registro contable/legal).

Campo	Tipo	Descripción
ID_Factura	varchar(10)	Llave primaria (ej. F000001).
nombre_cliente	varchar(30)	Nombre del cliente ("Consumidor Final" por defecto).
cod_empleado	varchar(10)	Cajero que realizó el cobro (FK a empleado).
ID_Pedido	varchar(10)	Pedido facturado (FK a pedido).
fecha_fac	date	Fecha de emisión.
impuesto	decimal(6,2)	Impuesto (15%), copiado del pedido al momento del cobro.
total	decimal(7,2)	Total facturado.
rtn_cliente	varchar(20)	RTN del cliente (opcional; validado con 13 o 14 dígitos).

gastos

Categorías de gasto del negocio (ej. Electricidad, Alquiler). Solo el administrador puede crear y editar categorías.

Campo	Tipo	Descripción
ID_gastos	varchar(10)	Llave primaria (ej. G000001).
nombre	varchar(20)	Nombre de la categoría.
tipo	varchar(10)	Publico u Operativo.
descripcion	varchar(30)	Descripción breve de la categoría.

gastos_detalle

Movimientos individuales (ocurrencias reales) de cada categoría de gasto. Disponible para cualquier rol con acceso al módulo. Cada movimiento ahora exige una breve descripción propia.

Campo	Tipo	Descripción
ID_detg	varchar(10)	Llave primaria (ej. GD000001).
fecha	date	Fecha del gasto.
monto	int(11)	Monto del movimiento.
ID_gastos	varchar(10)	Categoría a la que pertenece (FK a gastos).
descripcion	varchar(100)	Descripción del movimiento específico (obligatoria), ej. "factura de julio".

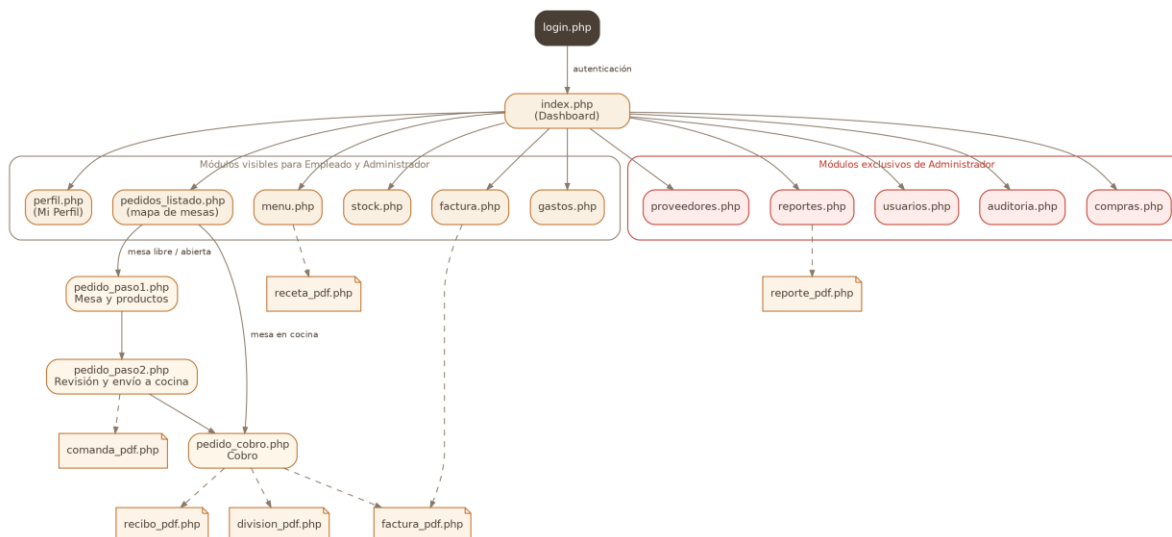
auditoria

Bitácora de solo lectura de acciones relevantes del sistema. Se alimenta desde includes/auditoria.php (función registrarAuditoria), llamada siempre después de que la acción correspondiente ya se ejecutó con éxito.

Campo	Tipo	Descripción
id	int(11)	Llave primaria autoincremental.
fecha_hora	datetime	Fecha y hora del evento (automática).
usuario	varchar(20)	Usuario que ejecutó la acción.
cod_empleado	varchar(10)	Empleado vinculado a ese usuario, si aplica.
modulo	varchar(30)	Módulo donde ocurrió el evento (ej. stock, pedido, usuarios).
accion	varchar(50)	Descripción corta de la acción (ej. "Producto editado").
detalle	varchar(255)	Detalle libre: qué cambió exactamente (valores antes/después cuando aplica).

7. Módulos del sistema

Esta sección describe cada pantalla del sistema: su propósito, quién puede usarla y las reglas de negocio relevantes identificadas en el código y sus comentarios.



7.1 Autenticación (login.php / logout.php / includes/auth.php)

Pantalla de acceso al sistema. Valida usuario y contraseña contra la tabla usuarios (con password_verify() y, por compatibilidad, comparación directa si la contraseña no está hasheada). Al autenticar correctamente:

- Se regenera el ID de sesión (session_regenerate_id(true)) por seguridad, para evitar fijación de sesión.
- Se guardan en \$_SESSION el ID de usuario, nombre de usuario, rol y código de empleado vinculado.
- Se registra el inicio de sesión exitoso (o el intento fallido, con el usuario intentado) en la auditoría.
- Se fuerza session_write_close() antes de redirigir, para asegurar que la sesión quede escrita en disco.

La marca del sistema en esta pantalla es puramente de texto (un círculo con "CASA LATINA" definido en CSS) — el sistema ya no depende de ningún archivo de imagen de logo. logout.php limpia por completo el arreglo \$_SESSION y destruye la sesión antes de volver al login.

7.2 Panel principal / Dashboard (index.php)

Página de inicio tras el login, visible para ambos roles pero con contenido distinto:

- Todos los roles ven: el estado de las mesas (libres, armando pedido, en cocina) y una alerta de productos con stock bajo (cantidad ≤ 5).
- Solo el administrador ve indicadores financieros del día (ventas, gastos, compras y utilidad neta) y una mini gráfica de barras con las ventas de los últimos 7 días.

El menú lateral y la tarjeta de sesión (usuario, rol, acceso a "Mi Perfil" y "Cerrar sesión") se arman en el propio index.php, filtrando los módulos según el rol. El total de mesas físicas está fijado en la variable

\$totalMesas = 12, valor que debe mantenerse sincronizado con la misma constante en pedidos_listado.php y pedido_paso1.php.

7.3 Mesas y pedidos activos (pedidos_listado.php)

Muestra un plano visual de mesas (con posiciones fijas configuradas en \$posicionesMesas, y una cuadrícula de respaldo automática para mesas sin posición definida) más el listado de pedidos de tipo Envío activos (si los hay) y el historial de pedidos cerrados del día.

Cada mesa cambia de color y de destino al hacer clic según su estado:

- Libre → lleva a pedido_paso1.php para iniciar un pedido nuevo en esa mesa.
- Armando (estado Abierto) → lleva de vuelta a pedido_paso1.php para seguir agregando productos.
- En cocina (estado EnCocina) → lleva directo a pedido_cobro.php para cobrar.

7.4 Pedido — Paso 1: Mesa y productos (pedido_paso1.php)

Primer paso del flujo de toma de pedidos. Ya no existe la opción de elegir entre "Mesa" o "Envío": todo pedido nuevo se guarda directamente como tipo Mesa en el código, sin preguntar nada al respecto.

- El número de mesa es un desplegable, no un campo de texto libre: se calcula en el servidor qué mesas están realmente ocupadas (\$mesasOcupadas) y solo se listan las libres, más la propia mesa si se está retomando un pedido en edición.
- Si se llega con ?mesa=N desde el mapa de mesas, se precarga ese número en el desplegable.
- Si se llega con ?id=ID_Pedido y ese pedido existe y sigue en estado Abierto, se recuperan sus productos para seguir editándolo; si no existe o ya cambió de estado, se trata como pedido nuevo.
- El campo "Cajero" quedó de solo lectura en pantalla (siempre muestra el propio usuario). El cambio real está en el servidor: ni este archivo ni pedido_paso2.php ni pedido_cobro.php leen ya el valor de un campo de formulario para el cajero — usan directamente \$_SESSION['cod_empleado'], así que no hay forma de hacerse pasar por otro empleado manipulando el navegador.
- Al continuar, se guarda el pedido con estado Abierto y todas sus líneas en pedido_detalle, dentro de una transacción; los totales se calculan y guardan en el Paso 2, no aquí.
- Desde aquí también se puede cancelar un pedido en edición (estado Abierto), lo que libera la mesa.

7.5 Pedido — Paso 2: Revisión y envío a cocina (pedido_paso2.php)

Segundo paso, con dos comportamientos distintos según el estado del pedido:

- Estado Abierto (primer envío): muestra la lista completa de productos con posibilidad de quitar ítems, calcula subtotal/impuesto/total en pantalla, y al confirmar ("Enviar a cocina") descuenta el stock de insumos según la receta de cada plato, recalcula los totales en el servidor y cambia el estado del pedido a EnCocina.
- Estado EnCocina (rondas adicionales): lo ya enviado queda fijo y de solo lectura; se puede agregar una tanda nueva de productos, que se guarda con un número de lote incrementado. También descuenta stock y recalcula totales.

La función descontarStockPorReceta() valida primero que haya suficiente stock para todos los insumos necesarios antes de descontar nada: si algún insumo no alcanza, lanza una excepción con el detalle de qué falta y no se descuenta nada (operación atómica).

La función `recalcularTotalesPedido()` siempre recalcula el subtotal desde cero sumando TODAS las líneas de `pedido_detalle` en la base de datos, nunca confía en un total enviado por el navegador.

Al enviar exitosamente a cocina se redirige a `pedido_cobro.php` indicando el número de lote recién enviado, lo que dispara la apertura automática de la comanda de esa ronda en una pestaña nueva.

7.6 Pedido — Paso 3: Cobro (`pedido_cobro.php`)

Pantalla de cobro, solo disponible cuando el pedido está en estado `EnCocina`. Aquí se define el método de pago, un descuento porcentual opcional, los datos del cliente (nombre y RTN opcionales) y el monto recibido en caso de pago en efectivo.

- El subtotal SIEMPRE se toma de la base de datos, nunca del formulario enviado por el navegador, para que nadie pueda manipular el descuento o el total desde el cliente.
- El RTN hondureño se valida con la función `rtnValido()`: debe tener 13 o 14 dígitos, sin contar guiones de formato.
- Con método Tarjeta se cobra el monto exacto (no hay concepto de cambio); con Efectivo se exige que el monto recibido sea mayor o igual al total, y se calcula el cambio.
- Al confirmar el cobro, dentro de una transacción: se actualiza el pedido a estado `Pagado`, y se genera automáticamente una factura ligada a ese pedido (nunca se crean facturas manualmente).
- Si se aplicó un descuento mayor a 0%, queda registrado explícitamente en la auditoría con el porcentaje y el monto.

La misma pantalla incluye una calculadora de "Dividir cuenta" (solo visual, no modifica la base de datos) y accesos para reimprimir la comanda completa o abrir el recibo en PDF una vez pagado.

7.7 Menú (`menu.php` / `receta_pdf.php`)

Catálogo de platos y bebidas disponibles para la venta. Los empleados solo tienen vista de lectura; solo el administrador puede crear, editar o desactivar ítems.

Cada ítem del menú tiene dos tipos de información asociada, independientes entre sí:

- Insumos de stock (tabla `menu_ingredientes`): qué productos de inventario y en qué cantidad se descuentan automáticamente al vender ese plato. Se gestionan con un buscador de insumos y se guardan como JSON antes de enviarse al servidor. Esta lista es la única fuente de ingredientes: se usa tanto para el descuento de inventario como para imprimir la receta — no hace falta escribirlos dos veces.
- Preparación de cocina (tabla `menu_receta`): texto libre con los pasos de preparación, tiempo estimado, porciones y notas — solo informativo para el personal de cocina, no afecta el inventario.

Reglas destacadas:

- No se permite crear dos ítems con el mismo ID de menú; se valida explícitamente antes de insertar, mostrando un mensaje claro en vez de un error de base de datos.
- Al editar, se compara el precio y la receta anteriores contra los nuevos para dejar constancia en la auditoría de qué cambió exactamente (precio, insumos de stock o preparación de cocina, cada uno por separado).
- Al eliminar: `menu_ingredientes` y `menu_receta` se borran en cascada automáticamente, pero `pedido_detalle` NO tiene esa cascada a propósito (para no perder la referencia de pedidos históricos). Si el plato ya se usó en algún pedido, el sistema detecta la violación de llave foránea y lo desactiva en vez de borrarlo.

Botón "Receta (PDF)": disponible para cualquier rol (incluido el empleado) junto a cada plato, abre `receta_pdf.php` en una pestaña nueva, que genera una ficha imprimible combinando el nombre, precio y descripción del plato, la lista de ingredientes de `menu_ingredientes` (con sus cantidades) y los pasos de preparación de `menu_receta`.

7.8 Stock / Inventario (`stock.php`)

Inventario general de insumos de cocina. Los empleados solo tienen vista de lectura; solo el administrador puede crear, editar o desactivar productos.

- La categoría se elige de un desplegable con 10 categorías estándar (Carnes y Embutidos, Lácteos, Verduras y Vegetales, Frutas, Granos/Cereales/Harinas, Condimentos y Especias, Bebidas, Panadería y Tortillas, Enlatados y Conservas, Desechables y Empaques), más "Otros" con un campo de texto libre para casos que no encajen.
- Permite buscar por nombre/ID y filtrar por proveedor.
- Resalta visualmente (fila roja, etiqueta "bajo") los productos con cantidad ≤ 5 , y muestra una tabla aparte de "productos con poco stock" con el proveedor a contactar.
- Al editar, si la cantidad o el precio cambiaron manualmente (fuera del flujo normal de Compras), queda registrado en la auditoría con los valores anterior y nuevo, como alerta de que alguien ajustó el inventario a mano.
- Al eliminar: compras y `menu_ingredientes` referencian producto; si el insumo ya se usó en algo, se desactiva en vez de borrarse, y el sistema avisa explícitamente si algún plato activo todavía lo usa en su receta.

7.9 Compras (`compras.php`)

Registro de compras de insumos a proveedores. Cada compra registrada:

- Genera un ID de compra correlativo (formato Cxxxxxx).
- Se conecta automáticamente con el stock: suma la cantidad comprada al campo `cantidad_pro` del producto correspondiente, dentro de la misma transacción.
- Permite elegir un proveedor específico para esa compra (independiente del proveedor por defecto asignado al producto en Stock); si no se especifica, se usa el proveedor asignado al producto al seleccionarlo en el buscador.
- La fecha queda fija al día de hoy: el campo se muestra de solo lectura en pantalla, y el servidor usa `date("Y-m-d")` propio en vez de leer cualquier valor que mande el formulario — así no se puede registrar una compra con fecha retroactiva o futura.

Incluye un buscador de productos en vivo que muestra precio de referencia y stock actual, y calcula el monto total sugerido ($\text{precio} \times \text{cantidad}$), editable manualmente antes de guardar.

7.10 Proveedores (`proveedores.php`)

Catálogo de proveedores, con búsqueda por nombre o ID. Para cada proveedor se muestra cuántos productos de Stock tiene asignados y el total histórico comprado.

- El formulario incluye correo electrónico como método de contacto además de teléfono y dirección.
- Al eliminar: `producto.ID_prov` referencia esta tabla; si algún insumo todavía le compra a este proveedor, se desactiva en vez de borrarse (mismo patrón que en Stock y Menú), informando cuántos y cuáles productos lo usan.

7.11 Gastos (gastos.php)

Pantalla dividida en dos mitades: categorías de gasto (arriba) y el detalle de movimientos de la categoría seleccionada (abajo).

- Crear una categoría nueva (ej. "Electricidad", "Alquiler") es exclusivo del administrador.
- Editar una categoría existente (nombre, tipo o descripción) también es exclusivo del administrador — cada fila tiene su botón "EDITAR" que carga los datos en el mismo formulario de alta.
- Registrar un movimiento (detalle) dentro de una categoría existente está disponible para cualquier rol con acceso al módulo, incluidos los empleados.
- Cada movimiento ahora exige una breve descripción (ej. "factura de julio") — el formulario no deja continuar sin ella, para saber después para qué fue exactamente ese pago.
- Cada categoría muestra su total acumulado (suma de todos sus movimientos).

7.12 Factura (factura.php / factura_pdf.php)

Las facturas nunca se crean ni se editan a mano: se generan automáticamente al cobrar un pedido en `pedido_cobro.php`. Este módulo es únicamente el historial: buscar (por cliente, número de pedido o rango de fechas), ver el detalle en pantalla e imprimir/descargar el PDF.

Por tratarse de un documento contable/legal, no se permite borrar facturas desde ningún punto del sistema.

`factura_pdf.php` genera el PDF completo con Dompdf: datos del cliente, del pedido (mesa), detalle de productos, subtotal, descuento (si aplica), impuesto, total y método de pago.

7.13 Reportes (reportes.php / reporte_pdf.php)

Módulo exclusivo de administrador que permite generar 9 tipos de reporte en PDF, eligiendo un rango de fechas (excepto el de inventario, que es una "foto" del momento actual):

Tipo de reporte	Contenido
Ventas	Listado de facturas emitidas en el rango, con total general.
Gastos	Movimientos de gastos del rango, agrupados por categoría con subtotales.
Compras	Compras a proveedores en el rango, con total general.
Compras por Proveedor	Las mismas compras, agrupadas por proveedor con subtotales.
Resumen Financiero	$\text{Ventas} - \text{Gastos} - \text{Compras} = \text{utilidad neta del período}$.
Platos Más Vendidos	Ranking de platos por cantidad vendida e ingreso generado, solo de pedidos ya Pagados.
Cierre de Caja	Separa el total facturado entre Efectivo y Tarjeta, con conteo de facturas por método y el efectivo esperado en caja.
Auditoría	Bitácora filtrable por usuario, módulo y texto libre, dentro del rango de fechas.
Inventario Actual	Foto del stock activo: cantidades, precios y valor total del inventario.

El formulario en `reportes.php` muestra u oculta dinámicamente los filtros adicionales (usuario/módulo/texto para auditoría, o la nota de que el rango de fechas no aplica para inventario) según el tipo elegido, y siempre envía la solicitud a `reporte_pdf.php`, que abre el PDF en una pestaña nueva.

7.14 Usuarios y Empleados (usuarios.php)

Módulo exclusivo de administrador con dos secciones independientes:

- Empleados: alta, edición, baja (o desactivación si tiene pedidos, facturas o una cuenta de usuario asociada) y reactivación del personal del restaurante. El puesto se elige de una lista estandarizada (Administrador, Gerente, Cajero, Mesero, Cocinero, Ayudante de Cocina, Bodega/Inventario, Repartidor, Limpieza) o "Otro" con texto libre.
- Usuarios: alta, edición y eliminación de cuentas de acceso al sistema, con asignación de rol y vínculo opcional a un empleado activo. Las contraseñas se guardan siempre con `password_hash()`; al editar, si el campo contraseña se deja en blanco, no se modifica la contraseña existente.

Incluye las protecciones descritas en la sección 5.2 para evitar dejar el sistema sin ningún administrador.

7.15 Mi Perfil (perfil.php)

Módulo nuevo, disponible para cualquier rol (Empleado o Administrador), que permite a cada usuario cambiar su propia contraseña sin depender de que un administrador entre al panel de Usuarios a hacerlo por él.

- Muestra los datos básicos de la cuenta en sesión: usuario, rol y código de empleado vinculado.
- El formulario de cambio de contraseña pide la contraseña actual (para confirmar identidad — se valida con `password_verify()`, con la misma compatibilidad de texto plano que usa `login.php`), la nueva contraseña (mínimo 6 caracteres) y su confirmación.
- Se rechaza si la contraseña actual es incorrecta, si la nueva tiene menos de 6 caracteres, si la confirmación no coincide, o si la nueva contraseña es igual a la actual.
- Este módulo no gestiona la contraseña de nadie más — solo la propia. Para restablecer la contraseña de otro usuario sigue siendo necesario el módulo Usuarios (solo Administrador).
- El cambio de contraseña queda registrado en la auditoría.

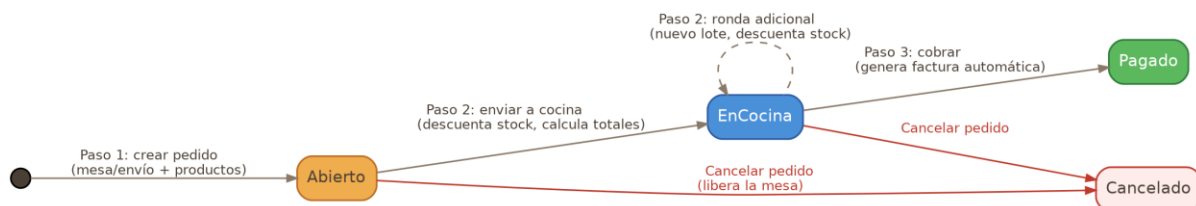
7.16 Auditoría (auditoria.php)

Bitácora de solo lectura, exclusiva de administrador, que muestra los últimos 300 movimientos que coincidan con los filtros aplicados (usuario, módulo, texto libre en acción/detalle, y rango de fechas). Los módulos disponibles en el filtro se obtienen dinámicamente de los valores ya registrados en la tabla (`SELECT DISTINCT modulo`).

Todos los módulos del sistema alimentan esta bitácora a través de la función `registrarAuditoria()` (`includes/auditoria.php`), que siempre se invoca después de que la operación correspondiente ya se ejecutó con éxito, nunca antes. Además de las creaciones/ediciones/eliminaciones normales, también quedan registrados los intentos bloqueados de un Empleado tratando de modificar algo restringido a Administrador (por ejemplo, intentar editar el menú o el stock directamente por POST).

8. Flujo completo de un pedido

El ciclo de vida de un pedido atraviesa cuatro estados posibles: Abierto → EnCocina → Pagado, con la posibilidad de pasar a Cancelado desde los dos primeros estados. A continuación se resume el recorrido típico de una mesa:



1. Se elige una mesa libre en el mapa de mesas (pedidos_listado.php), que abre pedido_paso1.php.
2. En el Paso 1 se agregan productos del menú, eligiendo la mesa de un desplegable que solo muestra mesas libres; el cajero se toma automáticamente de la sesión. Al continuar, se crea el pedido en estado Abierto junto con sus líneas en pedido_detalle (sin totales calculados todavía).
3. En el Paso 2 se revisa el pedido; al enviarlo a cocina se descuenta el stock de insumos según la receta de cada plato, se calculan subtotal/impuesto/total, y el pedido pasa a estado EnCocina. Se abre automáticamente la comanda en PDF de esa ronda.
4. Mientras el pedido sigue EnCocina, se pueden agregar rondas adicionales de productos desde el mismo Paso 2 (cada una descuenta stock y genera su propia comanda), sin afectar lo que ya se envió antes.
5. Cuando el pedido está listo para cerrarse, se pasa a pedido_cobro.php: se define método de pago, descuento y datos del cliente. El subtotal se recalcula siempre desde la base de datos.
6. Al confirmar el cobro, el pedido pasa a estado Pagado y se genera automáticamente una factura ligada a ese pedido. Desde ahí se puede imprimir el recibo en PDF.

En cualquier punto antes del cobro (estados Abierto o EnCocina), el pedido puede cancelarse, lo que libera la mesa y no genera factura.

9. Generación de documentos PDF

Todos los documentos imprimibles del sistema se generan con la librería Dompdf (paquete dompdf/dompdf), instalada vía Composer en la carpeta vendor/. El patrón es el mismo en los seis generadores: se captura el HTML del documento en un buffer de salida (ob_start() / ob_get_clean()), se carga en Dompdf y se transmite al navegador en tamaño carta, sin acceso remoto habilitado (isRemoteEnabled = false, por seguridad).

Archivo	Documento	Notas
factura_pdf.php	Factura	Trae el pedido y su detalle; recalcula nada, usa los valores ya guardados en factura y pedido.
recibo_pdf.php	Recibo de pedido	Incluye monto recibido y cambio; toma el nombre/RTN del cliente desde la factura asociada, si existe.
comanda_pdf.php	Comanda de cocina	Solo cantidades y nombres, sin precios. Admite imprimir todas las rondas (agrupadas por lote) o una sola ronda con ?lote=N, para reenvíos a media comida.
division_pdf.php	División de cuenta	Recalcula el descuento, impuesto y total en el servidor a partir del subtotal guardado (nunca confía en el total que mande el navegador) y reparte el total entre N personas, absorbiendo el redondeo en la Persona 1 para que la suma cuadre exacto.
reporte_pdf.php	Reportes financieros/operativos	Genera cualquiera de los 9 tipos de reporte descritos en la sección 7.13, según el parámetro tipo.

Archivo	Documento	Notas
receta_pdf.php	Ficha de receta de un plato	Combina los datos del plato (nombre, precio, descripción), la lista de ingredientes de menu_ingredientes con sus cantidades, y los pasos de preparación de menu_receta. Disponible para cualquier rol desde el módulo Menú.

includes/tema.php define, como constantes de PHP, la misma paleta de colores usada en assets/style.css, para que los documentos PDF mantengan la identidad visual del sistema sin depender de CSS externo (Dompdf no puede leer variables CSS del navegador).

10. Reglas de negocio transversales

Estas reglas se repiten de forma consistente en varios módulos del sistema y conviene entenderlas como principios de diseño generales:

10.1 Baja lógica en vez de eliminación física

Proveedores, productos (Stock), platos (Menú) y empleados nunca se eliminan de la base de datos si ya tienen historial asociado (compras, recetas, pedidos, facturas, etc.). En su lugar, el sistema detecta la violación de llave foránea (código de error PDO 23000) y marca el registro como activo = 0, conservando así la integridad del historial. El registro desactivado deja de aparecer como opción en los formularios de captura, pero sigue siendo consultable en los reportes e historiales existentes.

10.2 Recalculo de totales siempre en el servidor

Ningún monto que afecte dinero (subtotal, descuento, impuesto, total, división de cuenta) se toma del navegador. pedido_cobro.php, pedido_paso2.php y division_pdf.php recalculan siempre a partir de lo que hay guardado en la base de datos, precisamente para que nadie pueda manipular esos valores modificando el formulario o la URL desde el cliente.

10.3 Transacciones para operaciones multi-tabla

Toda operación que afecta más de una tabla relacionada (por ejemplo: registrar una compra y sumarla al stock, o enviar un pedido a cocina y descontar varios insumos a la vez) se envuelve en una transacción PDO (\$pdo->beginTransaction() / commit() / rollBack()), de forma que si algo falla a mitad de camino, no queda información inconsistente en la base de datos.

10.4 Auditoría después del hecho, nunca antes

La función registrarAuditoria() se invoca siempre después de que la operación correspondiente ya se ejecutó (y, cuando aplica, después del commit de la transacción), para que la bitácora refleje únicamente acciones que realmente ocurrieron. También se registran los intentos bloqueados de un rol tratando de hacer algo que no le corresponde.

10.5 Datos capturados desde la sesión, no desde el formulario

El caso del "cajero" en el flujo de pedidos es el ejemplo más claro: aunque en versiones anteriores era un campo de texto editable, ahora ningún paso del pedido lee ese dato de un campo de formulario — se toma siempre de \$_SESSION['cod_empleado']. El mismo principio aplica a la fecha de una compra, que el servidor fija con date("Y-m-d") propio en vez de aceptar la que mande el navegador.

10.6 Precios congelados en el momento de la venta

pedido_detalle.precio guarda el precio del plato en el momento exacto en que se agregó al pedido, independientemente de que después cambie el precio de ese plato en el catálogo del Menú. Esto asegura que pedidos y facturas históricas no se vean alterados por cambios de precio posteriores.

10.7 Categorías estandarizadas con opción de escape

Tanto la categoría de un producto de Stock como el puesto de un empleado se capturan con un desplegable de opciones estandarizadas, más una opción "Otro"/"Otros" que revela un campo de texto libre. Esto reduce la inconsistencia de datos escritos a mano ("bebida", "Bebidas", "BEBIDA") sin perder la flexibilidad de cubrir casos que no encajen en la lista.

11. Diseño visual

Todo el sistema comparte una única hoja de estilos (assets/style.css) que centraliza la paleta de colores en variables CSS (:root), de modo que cambiar el esquema de color de toda la aplicación es cuestión de editar un solo bloque de valores.

11.1 Paleta de colores

Variable	Valor	Uso
--color-primary	#DDA15E	Color de marca principal (botones, acentos, marca de texto)
--color-primary-dark	#BC6C25	Variante oscura (texto de marca, hover, barra superior)
--color-primary-light	#FAF0E1	Fondo suave con tinte de marca
--color-text-dark	#4A3F35	Texto principal
--color-text-muted	#8A7968	Texto secundario
--color-bg	#F5EBDD	Fondo general de la página
--color-mesa-libre	#5CB85C	Mesa libre (verde)
--color-mesa-armando	#F0AD4E	Mesa armando pedido (naranja)
--color-mesa-cocina	#4A90D9	Mesa en cocina (azul)

includes/tema.php replica exactamente los mismos valores como constantes de PHP, para que los documentos PDF generados con Dompdf usen la misma identidad visual.

11.2 Marca del sistema

A diferencia de versiones anteriores, el sistema ya no depende de ningún archivo de imagen para mostrar su marca: tanto en el login como en el encabezado del panel se usa un elemento .marca de puro CSS (un círculo con borde de color primario y el texto "CASA LATINA" en el centro). Esto elimina la dependencia de un archivo assets/logo.png y evita que la marca se rompa visualmente si ese archivo llegara a faltar.

11.3 Estructura visual común

- Barra superior (topbar) con el nombre del módulo actual y enlace de regreso al inicio.
- Menú lateral (sidebar) generado dinámicamente a partir de la lista de módulos, filtrando los que no correspondan al rol del usuario en sesión.

- Tarjeta de sesión en la parte inferior del menú lateral, con avatar (inicial del usuario), nombre, rol, enlace a "Mi Perfil" y enlace para cerrar sesión.
- Sistema de tarjetas (.pd-card) y filas en cuadrícula pareja (.pd-row / .pd-field) reutilizado en todos los módulos, con botones de acción visualmente consistentes.

12. Requisitos e instalación

12.1 Requisitos

- PHP con extensión PDO para MySQL/MariaDB.
- Servidor MariaDB/MySQL (probado con MariaDB 11.8).
- Composer, para instalar la dependencia dompdf/dompdf (versión ^3.1, según composer.json).

12.2 Variables de entorno de conexión (includes/db.php)

La conexión a la base de datos se configura mediante variables de entorno, con valores por defecto para desarrollo local:

```
DB_HOST → host de la base de datos (por defecto: localhost)
DB_NAME → nombre de la base de datos (por defecto: casalatinasmartsystem)
DB_USER → usuario de la base de datos (por defecto: root)
DB_PASS → contraseña del usuario (por defecto: 1234)
```

12.3 Pasos de instalación

1. Crear la base de datos ejecutando el esquema: `sudo mariadb -u root -p casalatinasmartsystem < FCLschema.sql`.
2. Cargar los datos de prueba: `sudo mariadb -u root -p casalatinasmartsystem < datos_prueba.sql` (proveedores, insumos, empleados, dos usuarios de prueba, menú con recetas, una compra y un pedido histórico con su factura).
3. Instalar las dependencias de Composer: `composer install` (crea la carpeta vendor/ con Dompdf).
4. Configurar las variables de entorno de conexión si difieren de los valores por defecto.
5. Servir el proyecto con un servidor web con soporte PHP (ej. Apache, Nginx + PHP-FPM, o el servidor embebido de PHP) apuntando a la raíz del proyecto.
6. Iniciar sesión con alguno de los usuarios de prueba (usuario admin o caja, contraseña 1234) y comenzar a operar.

Nota: los usuarios de ejemplo (admin / caja, ambos con contraseña 1234) deben eliminarse o cambiarse antes de usar el sistema en un entorno de producción real.

13. Pruebas del sistema

Para validar el funcionamiento y la experiencia de uso de Casa Latina Smart System se aplicaron dos métodos de prueba complementarios: primero un ad hoc testing con criterio, realizado por una persona con conocimiento del proyecto que recorrió cada módulo evaluándolo contra buenas prácticas de usabilidad y consistencia funcional; y después una prueba de caja negra, realizada por personas externas al desarrollo, sin ninguna explicación previa de cómo usar el sistema.

13.1 Ad hoc testing con criterio

El ad hoc testing es una técnica de prueba informal, sin un guion ni casos de prueba preestablecidos: quien prueba explora el sistema con libertad, simulando el uso real de un usuario. En este proyecto se aplicó con criterio, es decir, guiando esa exploración libre con un conjunto de principios de calidad de software y experiencia de usuario (consistencia entre pantallas, claridad de los controles, coherencia entre roles, validación de datos, mensajes de error apropiados, entre otros), documentando cada hallazgo en el momento en que se encontró.

Esta prueba se realizó recorriendo todos los módulos del sistema desde la vista de administrador. A continuación se resumen las observaciones y oportunidades de mejora detectadas por módulo:

Módulo	Observaciones detectadas
Pedidos	El acceso a "Nuevo pedido" es solo un enlace de texto y no un botón propio, lo que reduce su visibilidad; se sugiere que el número de mesa se seleccione con un menú desplegable en vez de escribirse manualmente; y en un intento no fue posible avanzar el flujo de captura de productos hasta el siguiente paso.
Menú	No queda claro por qué el sistema permite reactivar un producto desactivado sin ninguna restricción de tiempo; en un intento no fue posible guardar la receta de un plato pese a tener los datos completos; falta un botón visible para limpiar el formulario; el ID del ítem se captura manualmente en un campo de texto (varchar), lo que permite combinaciones de letras y números o duplicar accidentalmente un producto; y un mensaje de error al intentar registrar un ítem ya existente llegó a mostrar parte del código interno.
Factura	No fue posible completar pruebas directas sobre este módulo, ya que el flujo de creación de pedido no pudo finalizarse en ese momento; adicionalmente, no quedó claro si el documento corresponde a la venta del restaurante al cliente o a una compra a proveedor.
Stock	Se sugiere manejar una lista fija de categorías de producto en vez de un campo de texto libre, para reducir errores de captura y facilitar la búsqueda de insumos por categoría; se observó un caso en el que el stock no se descontó al preparar un plato con receta configurada; y desde la vista de empleado fue posible desactivar un producto, permiso que se esperaría reservado al administrador.
Compras	En algunos campos del formulario no fue posible escribir directamente; y se cuestionó si la fecha de la compra debería ser de solo lectura (fecha actual) o limitarse a un rango acotado, para evitar que se registre con una fecha retroactiva editada manualmente.
Proveedores	Se sugiere ubicar el formulario de "agregar proveedor" en la parte superior de la pantalla y dejar la búsqueda debajo; se identificó un error al dejar vacío el campo de dirección; y se sugiere permitir más de un teléfono, dirección o correo electrónico por proveedor.
Gastos	El sistema permite registrar una categoría de gasto sin descripción; se sugiere reestructurar el módulo para que los movimientos se capturen directamente dentro de categorías estandarizadas, evitando tener que buscar la categoría y especificar el monto por separado.
Usuarios	Se sugiere que el campo "puesto" del empleado sea un menú desplegable con las áreas de trabajo predefinidas, en vez de un campo de texto libre, para reducir errores de asignación de área y de rol.
Auditoría	No se identificaron problemas ni dudas sobre su funcionamiento.

Nota: varias de las observaciones anteriores corresponden específicamente a la vista de administrador; al repetir el recorrido en la vista reducida de empleado, las mismas funciones se comportaron correctamente.

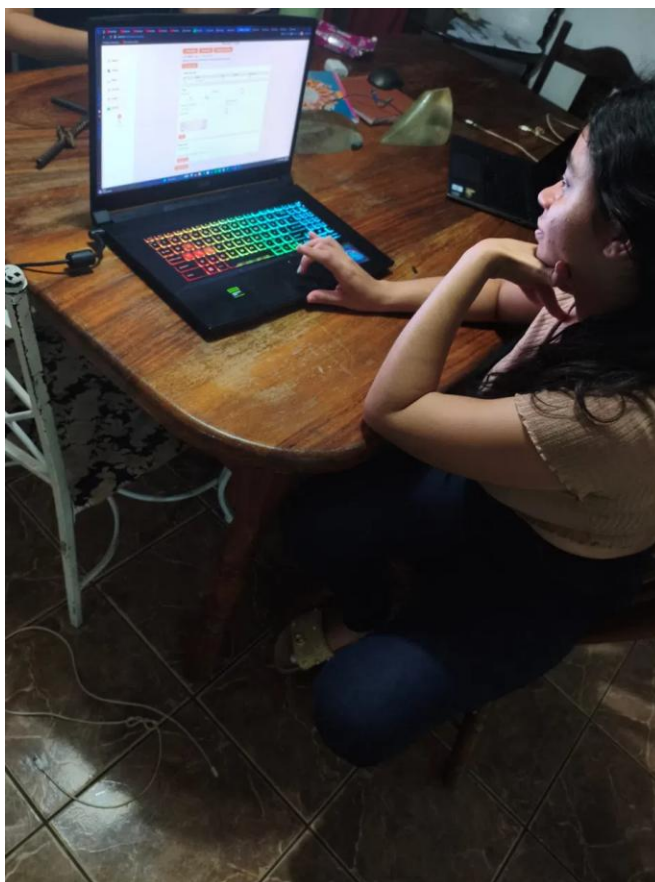
Esta prueba se realizó sobre una versión anterior del sistema — varios de los puntos señalados (mesa por menú desplegable, categorías estandarizadas en Stock, puesto estandarizado en Usuarios, descripción obligatoria en los movimientos de gasto, entre otros) ya fueron corregidos en la versión actual descrita en este documento.

13.2 Pruebas de caja negra

La prueba de caja negra evalúa el sistema únicamente a partir de sus entradas y salidas observables, sin conocimiento de su código o su estructura interna. Para este proyecto se realizaron dos pruebas de caja negra con personas ajenas al desarrollo, a quienes no se les explicó de antemano cómo usar el sistema: su único conocimiento previo era que se trataba de un sistema para un restaurante. Esta condición asegura que los resultados reflejan la experiencia real de una persona nueva frente al sistema, sin ningún sesgo de conocimiento técnico o funcional previo.

Caso de prueba 1

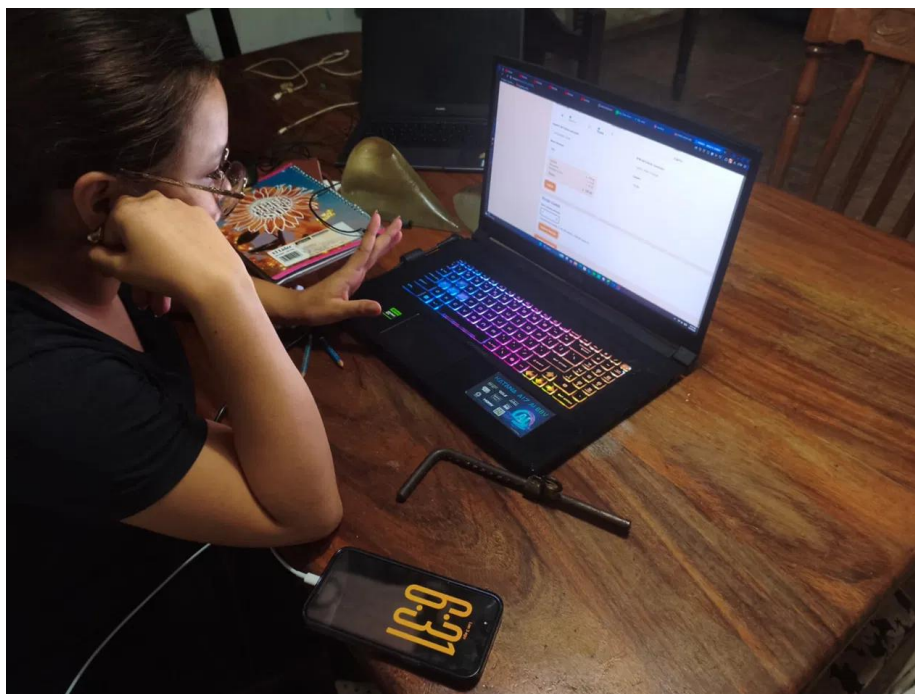
Campo	Detalle
Tarea asignada	Armar y registrar un pedido nuevo, seleccionando productos del menú.
Conocimiento previo del evaluador	Ninguno sobre el manejo del sistema; solo sabía que se trataba de un sistema para un restaurante.
Resultado esperado	Completar la tarea asignada navegando el sistema por cuenta propia, sin errores que impidan concluirla.
Resultado obtenido	No se encontraron fallas. La evaluadora completó la tarea sin inconvenientes.
Observaciones cualitativas	Comentó sentirse muy cómoda utilizando el sistema, e incluso lo encontró entretenido; lo calificó como fácil de navegar.



Evidencia — Caso de prueba 1

Caso de prueba 2

Campo	Detalle
Tarea asignada	Completar el proceso de cobro de un pedido (método de pago, datos del cliente y total a pagar).
Conocimiento previo del evaluador	Ninguno sobre el manejo del sistema; solo sabía que se trataba de un sistema para un restaurante.
Resultado esperado	Completar la tarea asignada navegando el sistema por cuenta propia, sin errores que impidan concluirla.
Resultado obtenido	No se encontraron fallas. La evaluadora completó la tarea sin inconvenientes.
Observaciones cualitativas	Comentó sentirse muy cómoda utilizando el sistema, e incluso lo encontró entretenido; lo calificó como fácil de navegar.



Evidencia — Caso de prueba 2

13.3 Conclusión de las pruebas de caja negra

En ambos casos, evaluadoras sin ningún conocimiento previo del sistema lograron completar la tarea asignada sin encontrar fallas. El resultado positivo, sumado a la percepción de comodidad y facilidad de navegación reportada, es un indicio favorable de la usabilidad general del sistema desde la perspectiva de un usuario nuevo, en contraste con las oportunidades de mejora más específicas identificadas durante el ad hoc testing con criterio, orientado a un análisis más profundo de consistencia y buenas prácticas.

14. Reporte de Cambios — Antes vs. Ahora, por función

Esta sección compara la función original de cada módulo contra su estado actual, junto con el mecanismo detrás de cada cambio. No es un historial paso a paso de cada sesión de trabajo, sino qué hacía antes el sistema, qué hace ahora, y por qué se hizo el cambio.

14.1 Pedidos

Aspecto	Antes (original)	Ahora (actual)
Formulario	Un solo archivo (pedido.php) mezclaba búsqueda de productos, carrito, cálculo de totales, cobro y cancelación en una sola pantalla larga. No había pasos ni forma de saber en qué punto estaba un pedido.	Flujo de 3 pasos en archivos separados: pedido_paso1.php (armar), pedido_paso2.php (revisar y enviar a cocina), pedido_cobro.php (cobrar). Cada uno valida el estado real del pedido en la base de datos antes de dejar avanzar — no se puede cobrar un pedido que sigue "Abierto", por ejemplo.
Mesa	No existía ningún mapa de mesas. El número de mesa era un campo de texto libre: se podía escribir cualquier número, repetido o inexistente, sin ninguna validación.	pedidos_listado.php dibuja un plano del restaurante con cada mesa como un círculo de color (verde=libre, naranja=armando, azul=en cocina). Al crear un pedido nuevo, el campo de mesa es un desplegable que consulta qué mesas están realmente ocupadas y solo muestra las libres (más la propia mesa, si se está retomando un pedido en edición).

Aspecto	Antes (original)	Ahora (actual)
Tipo de pedido	Cada pedido preguntaba si era "Mesa" o "Envío", con un selector que mostraba/ocultaba el campo de número de mesa según la opción elegida.	Se quitó el selector por completo — en la práctica casi nunca se usaba "Envío", así que ahora todo pedido nuevo se guarda como "Mesa" directamente en el código, sin preguntar nada.
Cajero	Era un campo de texto normal dentro del formulario. El valor que mandara el navegador se guardaba tal cual — cualquiera podía escribir cualquier código de empleado, incluso uno que no fuera el suyo, o editar el HTML para mandar otro.	El campo quedó de solo lectura en pantalla (siempre muestra el propio usuario). Pero el cambio real está en el servidor: pedido_paso1.php, pedido_paso2.php y pedido_cobro.php ya ni siquiera leen el campo "cajero" del formulario — usan directamente <code>\$_SESSION['cod_empleado']</code> , así que no hay forma de hacerse pasar por otro empleado aunque se manipule el navegador.
Inventario	No existía ninguna relación entre vender un plato y el stock. Vender 50 baleadas no bajaba ni una tortilla del inventario.	Al enviar un pedido a cocina, el sistema revisa la receta de cada plato (tabla menu_ingredientes), suma cuánto insumo hace falta en total, verifica que haya suficiente stock de cada uno, y si alcanza, lo descuenta automáticamente. Si no alcanza, no deja enviar el pedido y dice exactamente qué insumo falta y cuánto.
Rondas adicionales	Una vez guardado el pedido, no había forma de agregarle nada más — si el cliente pedía algo más a media comida, había que crear un pedido nuevo aparte.	Se puede volver a un pedido que ya está "En cocina" y agregar más productos. Cada envío queda marcado con un número de "ronda" (columna lote en pedido_detalle), así que la comanda de una ronda nueva solo muestra lo recién agregado — cocina no ve otra vez lo que ya está preparando.
Comanda de cocina	No existía ningún documento para cocina — el mesero tenía que decir de viva voz o a mano qué se pidió.	comanda_pdf.php genera un ticket en PDF sin precios (solo cantidad y nombre de cada producto, letra grande) que se abre automáticamente en una pestaña nueva cada vez que se envía un pedido o una ronda a cocina.
Cobro	Solo aceptaba efectivo, con un campo de monto recibido y cálculo de cambio. No había forma de aplicar descuentos, cobrar con tarjeta, ni se generaba ningún documento para el cliente.	Método de pago Efectivo o Tarjeta (con tarjeta se cobra el monto exacto, sin campo de cambio); descuento porcentual que recalcula todo en pantalla; nombre y RTN del cliente opcionales, con validación de formato de RTN; calculadora de división de cuenta entre varias personas; y al cobrar se genera sola la factura y queda disponible el recibo en PDF — nada de esto existía antes.
Cancelación	No había ningún botón ni flujo para cancelar un pedido ya iniciado.	Disponible desde los 3 pasos del pedido, con confirmación. Cancela el pedido, libera la mesa de inmediato en el mapa, y queda registrado en Auditoría quién canceló y cuándo.

14.2 Menú

Aspecto	Antes (original)	Ahora (actual)
Datos básicos	Formulario simple con nombre, precio, tipo y descripción. El ID del plato se escribía a mano en cada alta, sin ninguna verificación de que no existiera ya.	El ID se genera solo (M001, M002...) y además, antes de insertar, se hace una consulta explícita para avisar con un mensaje claro si el ID ya existe, en vez de que el usuario se tope con un error de base de datos.
Eliminar	El botón "Eliminar" intentaba un DELETE directo. Si el plato ya se había vendido alguna vez, la base de datos rechazaba el borrado por la restricción de llave foránea con pedido_detalle, y el usuario veía un error críptico de MySQL en pantalla.	Se intenta borrar; si la base de datos lo rechaza (código de error 23000), en vez de mostrar el error crudo, el sistema marca el plato como inactivo automáticamente y explica por qué. Un plato inactivo desaparece de las búsquedas para pedidos nuevos, pero los pedidos viejos que ya lo usaron se siguen viendo sin problema. Se puede reactivar en cualquier momento.
Receta / insumos de stock	El campo menu.ID_Producto permitía vincular un plato a un único producto de stock, sin cantidad — en la práctica no servía para nada real, ya que un plato normalmente usa varios insumos en cantidades distintas.	Tabla nueva menu_ingredientes: relación de varios-avarios entre plato e insumo, cada uno con su cantidad_necesaria. Se arma con un buscador (nombre o ID del insumo + cantidad). Esta misma lista es la que se usa tanto para descontar stock al vender como para

Aspecto	Antes (original)	Ahora (actual)
		mostrarse en la ficha de receta imprimible — no hay que escribir los ingredientes dos veces en ningún lado.
Receta de cocina	No existía ningún lugar para anotar cómo se prepara un plato.	Tabla nueva menu_receta con pasos de preparación, tiempo estimado y porciones, y notas — información que no tiene que ver con el stock, es para el cocinero. Cada plato tiene un botón "Receta (PDF)" que genera una ficha imprimible combinando esta información con la lista de insumos.
Permisos	Cualquiera con acceso al módulo Menú (Empleado o Administrador) podía crear, editar o borrar platos libremente.	Solo Administrador puede modificar — el formulario de alta/edición y los botones de acción ni siquiera se muestran si no eres Administrador, y el servidor también rechaza la operación aunque alguien intente mandar el POST directo. Empleado ve la tabla completa, pero en modo solo lectura.

14.3 Stock / Inventario

Aspecto	Antes (original)	Ahora (actual)
Categoría	Campo de texto libre — cada quien escribía lo que se le ocurriera ("bebida", "Bebidas", "BEBIDA", etc.), sin ningún control de consistencia.	Desplegable con 10 categorías estándar (Carnes y Embutidos, Lácteos, Verduras y Vegetales, Frutas, Granos/Cereales/Harinas, Condimentos y Especies, Bebidas, Panadería y Tortillas, Enlatados y Conservas, Desechables y Empaques) más "Otros" con un campo de texto libre para casos que no encajen. La columna en la base de datos también se ensanchó (de VARCHAR(10) a VARCHAR(50)) porque los nombres nuevos no cabían en el largo original.
Cantidades	La columna cantidad_pro era de tipo INT — solo números enteros. Cualquier receta o compra con una cantidad fraccionaria (ej. 0.15 lb de pollo) se redondeaba silenciosamente al guardar, sin ningún aviso.	Migrada a DECIMAL(10,2) — admite fracciones reales. Esto era imprescindible para que el descuento automático de stock por receta funcionara correctamente con insumos que se miden por peso.
Eliminar	Igual que en Menú: tronaba con un error de base de datos si el producto ya se había usado en alguna compra o receta.	Se desactiva en vez de romperse, y si algún plato activo todavía depende de ese insumo en su receta, el mensaje lo dice explícitamente (con los nombres de los platos), para decidir si hace falta actualizar esa receta.
Proveedor	Existía un campo opcional para asignar proveedor a un producto, pero no se mostraba en ningún listado ni se podía buscar por él.	Columna "Proveedor" visible en la tabla principal; filtro de búsqueda por proveedor (combinable con la búsqueda por texto); y en la alerta de "poco stock" aparece el nombre y teléfono del proveedor de cada producto bajo, para saber a quién llamar sin salir del módulo.
Permisos	Cualquiera con acceso podía crear, editar o desactivar productos.	Solo Administrador edita; Empleado ve la tabla completa (incluyendo la alerta de poco stock) en modo solo lectura, igual que en Menú.

14.4 Compras

Aspecto	Antes (original)	Ahora (actual)
Producto	Un <select> de HTML mostrando "ID - nombre" de todos los productos, sin ningún dato adicional visible.	Buscador con resultados en vivo que muestra ID, nombre, precio de referencia y stock actual de cada producto, para decidir con más información antes de elegir.
Proveedor de la compra	No existía — una compra solo sabía qué producto se compró, nunca a quién.	Columna nueva compras.ID_prov. Al elegir el producto, el proveedor se autocompleta con el que tiene asignado en Stock, pero se puede cambiar manualmente si esa vez se compró a alguien más. Esto es lo que permite después ver "cuánto le he comprado a cada proveedor" en el módulo Proveedores y en Reportes.

Aspecto	Antes (original)	Ahora (actual)
Fecha	Campo de fecha editable libremente, sin ningún límite — se podía registrar una compra con cualquier fecha pasada o futura.	El campo quedó fijo al día de hoy (readonly en pantalla), y el INSERT en el servidor usa date("Y-m-d") del propio servidor en vez de leer el valor que mande el formulario — así nadie puede "atrasar" el registro de una compra para evadir responsabilidad por un desabasto.
ID de compra	Escrito a mano.	Autogenerado (C000001, C000002...).
Cantidades	Columna INT, mismo problema que en Stock: no admitía compras fraccionarias.	Migrada a DECIMAL(10,2), igual que producto.cantidad_pro.

14.5 Proveedores

Aspecto	Antes (original)	Ahora (actual)
Conexión con el resto del sistema	Módulo aislado — ningún otro módulo mostraba ni usaba esta información más allá de un campo opcional en Stock.	La tabla de Proveedores ahora muestra, para cada uno, cuántos productos de Stock tiene asignados y cuánto se le ha comprado en total (monto y número de compras), calculado en vivo sobre la tabla compras.
Contacto	Solo teléfono y dirección.	Se agregó correo electrónico (email_prov) como método de contacto alternativo, para proveedores sin teléfono fijo.
Eliminar	Tronaba con un error de base de datos si el proveedor tenía productos asignados.	Se desactiva en vez de romperse, avisando cuántos y cuáles productos todavía lo tienen asignado.
Teléfono	Columna tel_prov de tipo INT — rechazaba números con guion (formato típico hondureño, ej. 9950-1113) y también rechazaba dejar el campo vacío.	Migrada a VARCHAR(20) — acepta cualquier formato, incluyendo vacío.

14.6 Factura, Recibo, Comanda y División de Cuenta

Aspecto	Antes (original)	Ahora (actual)
Generación de factura	Formulario aparte donde se escribía a mano el número de factura, el nombre del cliente, el cajero y el número de pedido — sin validar que el pedido existiera, que ya no tuviera factura, ni que los montos coincidieran con el pedido real.	Se genera sola, dentro de la misma transacción del cobro (pedido_cobro.php) — nunca se llena a mano. Esto elimina por completo el riesgo de facturas duplicadas, montos mal copiados, o facturas huérfanas sin pedido real detrás.
Historial de facturas	No existía ninguna pantalla para verlo — factura.php era solo el formulario de alta, sin ninguna tabla debajo.	factura.php se convirtió en un historial buscable por cliente, número de pedido o rango de fechas, con vista de detalle en pantalla y en PDF por cada una.
Borrar factura	El formulario original tenía botón de borrar.	Se quitó esa opción por completo — una factura es un documento contable, y permitir borrarla dejaría el historial de ventas incompleto.
"Imprimir"	El único botón de imprimir llamaba a window.print() del navegador sobre la pantalla completa — salía el menú de navegación, la URL de la página, y el contenido ocupaba solo la esquina superior izquierda de la hoja.	3 documentos PDF reales generados con la librería dompdf: recibo (recibo_pdf.php), comanda de cocina (comanda_pdf.php) y desglose de división de cuenta (division_pdf.php). Todos con membrete propio, tamaño carta completo, y sin ningún elemento del navegador.
División de cuenta	No existía.	Calculadora en la pantalla de cobro: se indica entre cuántas personas se reparte, y muestra cuánto le toca pagar a cada una (reparto parejo sobre el total ya con descuento aplicado). Se puede imprimir ese desglose en PDF.

14.7 Gastos

Aspecto	Antes (original)	Ahora (actual)
Pantallas	Dos archivos separados: gastos.php (solo categorías) y detalle_gasto.php (solo movimientos, con un select para elegir la categoría). Para ver el detalle de una categoría había que cambiar de página.	Una sola vista maestro-detalle en gastos.php: las categorías arriba (con su total acumulado), y al hacer clic en una, su detalle de movimientos aparece abajo en la misma página.
Tipo de categoría	Público, Privado u Operativo — sin que quedara claro en ningún lado qué significaba cada uno.	Se quitó "Privado" por no tener un propósito claro; quedan Público y Operativo.
Editar categoría	No existía ninguna forma de corregir el nombre, tipo o descripción de una categoría ya creada — solo se podían crear nuevas.	Cada categoría tiene su botón "Editar" (solo Administrador), que carga sus datos en el mismo formulario de alta y lo convierte en modo edición.
Descripción del movimiento	Un movimiento (gastos_detalle) solo tenía fecha y monto — no había forma de saber, meses después, para qué fue exactamente ese pago.	Se agregó una columna descripción a gastos_detalle, y el formulario exige llenarla (ej. "factura de julio", "reparación de refrigerador") antes de dejar registrar el movimiento.

14.8 Reportes

Aspecto	Antes (original)	Ahora (actual)
Selección de reporte	Un formulario con un <select> de 3 opciones (ventas, gastos, compras) y un rango de fechas.	Cuadrícula de 9 tarjetas seleccionables, con una breve descripción de qué muestra cada una, más campos de filtro adicionales que aparecen solo cuando el tipo elegido los necesita (por ejemplo, los filtros de usuario/módulo/texto solo aparecen si se elige Auditoría).
Tipos de reporte	3: Ventas, Gastos, Compras — cada uno mostrando una tabla cruda con los nombres de columna de la base de datos tal cual (ID_Factura, fecha_fac, etc.), sin ningún procesamiento.	9 tipos, cada uno con su propia lógica de agregación: Ventas, Gastos (agrupado por categoría con su descripción), Compras, Compras por Proveedor (agrupado por proveedor), Resumen Financiero (Ventas – Gastos – Compras = utilidad neta), Platos Más Vendidos (ranking por cantidad e ingreso), Cierre de Caja (Efectivo vs. Tarjeta, con el efectivo esperado en caja), Auditoría (con filtros) e Inventario Actual (valor total del stock, no depende de fechas).
Formato de salida	La tabla se mostraba directo en la página del navegador, sin ninguna opción de imprimir o guardar con formato.	Cada reporte se genera como PDF con membrete propio (reporte_pdf.php), listo para imprimir o archivar.

14.9 Dashboard (Inicio)

Aspecto	Antes (original)	Ahora (actual)
Contenido general	index.php mostraba solo una cuadrícula de botones de acceso a cada módulo, sobre una página vacía sin ninguna información adicional.	Layout de dos columnas: un sidebar con los botones de los módulos disponibles según el rol (con scroll si no caben) y una tarjeta de sesión al fondo (usuario, rol, acceso a "Mi Perfil" y "Cerrar sesión"); y un panel principal con información real del negocio.
Indicadores financieros	No existían.	Solo para Administrador: ventas, gastos, compras y utilidad neta del día, calculados en vivo con consultas SQL agregadas; más una gráfica de barras (CSS puro, sin librerías) de las ventas de los últimos 7 días, con un tooltip que muestra el monto exacto al pasar el mouse.
Información operativa	No existía.	Visible para cualquier rol: cuántas mesas están libres/armando/en cocina ahora mismo (con link al mapa), y los 5 productos con menor stock (con link al inventario completo).

14.10 Usuarios, Mi Perfil y Auditoría (módulos nuevos)

Aspecto	Antes (original)	Ahora (actual)
Crear usuarios y empleados	No existía ninguna forma de hacerlo desde la aplicación — había que insertar las filas directo en la base de datos, incluyendo calcular a mano el hash de la contraseña.	Módulo usuarios.php (solo Administrador): primero se crea el Empleado (nombre, puesto por desplegable estandarizado + "Otro", salario), luego se le vincula una cuenta de Usuario (login, contraseña con password_hash(), rol).
Protección contra quedarse sin administrador	No existía ningún resguardo — se podía, sin querer, dejar el sistema sin ningún usuario con rol de administrador, o borrar la propia cuenta mientras se estaba usando.	El sistema cuenta cuántos usuarios con rol='administrador' existen antes de permitir eliminar o quitarle el rol a uno, y bloquea la acción si sería el último. Tampoco deja eliminar la propia cuenta mientras se tiene sesión abierta.
Cambiar la propia contraseña	Ningún usuario podía cambiar su propia clave — dependía por completo de que un Administrador entrara al panel de Usuarios y la cambiara por él.	Módulo nuevo perfil.php ("Mi Perfil"), disponible para cualquier rol: pide la contraseña actual para confirmar identidad, exige mínimo 6 caracteres y que coincida con la confirmación.
Registro de acciones (Auditoría)	No existía ningún rastro de quién hacía qué en el sistema.	Tabla auditoria + módulo de solo lectura, filtrable por usuario/módulo/texto/fecha. Registra: login (exitoso y fallido), cancelación de pedidos y descuentos aplicados, creación/edición/eliminación en Menú, Stock, Proveedores, Compras, Gastos y Usuarios, cambios de precio y de receta (separando insumos de stock vs. receta de cocina) en Menú, cambios manuales de cantidad/precio en Stock, y los intentos bloqueados de un Empleado tratando de modificar algo restringido a Administrador.

14.11 Diseño Visual y Seguridad

Aspecto	Antes (original)	Ahora (actual)
Esquema de colores	Naranja saturado con fondo en degradado y barra superior negra, con los códigos de color escritos directo en cada hoja de estilo — cambiar el color de algo significaba editarlo archivo por archivo.	assets/style.css centraliza la paleta en variables CSS (bloque :root), e includes/tema.php define las mismas constantes en PHP para los generadores de PDF (dompdf no soporta variables CSS de forma confiable). Cambiar el esquema completo del sistema es editar esos 2 archivos.
Maquetado de las pantallas	Cada módulo usaba <div class="fila"> apilados verticalmente, sin ninguna alineación consistente entre campos.	Sistema de tarjetas (.pd-card) y filas en cuadrícula pareja (.pd-row / .pd-field) reutilizado en todos los módulos, con botones de acción visualmente consistentes (.btn / .btn-link) en vez de mezclar texto plano y botones.
IDs de registros	Se escribían a mano en Pedidos, Compras, Gastos, Proveedores y Usuarios — con riesgo real de que dos personas usaran el mismo ID, o de exceder el largo de la columna (esto pasó literalmente con ID_Pedido, definida como VARCHAR(10), que se desbordaba con IDs basados en fecha completa).	Autogenerados por conteo en todos los módulos (P000001, C000001, G000001, PV0001, E001, U001, etc.), eliminando el riesgo de choque manual.
Totales de venta	El total que mandaba el formulario del navegador al cobrar se guardaba tal cual en la base de datos — alguien con conocimientos básicos podía editar el HTML de la página y mandar un total distinto al real.	El servidor siempre recalcula subtotal, descuento, impuesto y total a partir del subtotal real guardado en la base de datos al momento de enviar el pedido a cocina — nunca se confía en ningún número que llegue del navegador.
Operaciones con varias tablas	No usaban transacciones — si algo fallaba a la mitad (ej. se guardaba el pedido pero fallaba el detalle), quedaban datos inconsistentes sin ninguna forma de deshacerlos.	Cobrar un pedido, enviarlo a cocina, registrar una compra y guardar una receta están envueltos en transacciones (beginTransaction/commit/rollback) — si algo falla, no se guarda nada a medias.

15. Diagrama de Gantt del Proyecto

El siguiente diagrama muestra la planificación y el avance real del proyecto a lo largo del tiempo, organizado por tarea, responsable y estado, con los hitos principales (Inicio del Proyecto, Cambio de Arquitectura,

Creación de GitHub y Presentación) marcados por separado del resto de las tareas. Se incluye en la página siguiente, en formato horizontal, para que se pueda leer con más detalle.

